

COMP 576 Assignment 0 Report

Task 1:

```
active environment : base
active env location : /opt/anaconda3
    shell level : 1
    user config file : /Users/keanu/.condarc
populated config files : /opt/anaconda3/.condarc
                        /opt/anaconda3/condarc.d/anaconda-auth.yml
    conda version : 26.5.3
conda-build version : 26.5.0
    python version : 3.14.6.final.0
        solver : libmamba (default)
virtual packages : __archspec=1=m4
                  __conda=26.5.3=0
                  __osx=26.6.2=0
                  __unix=0=0
base environment : /opt/anaconda3 (writable)
conda av data dir : /opt/anaconda3/etc/conda
conda av metadata url : None
channel URLs :
https://repo.anaconda.com/pkgs/main/osx-arm64
https://repo.anaconda.com/pkgs/main/noarch
https://repo.anaconda.com/pkgs/r/osx-arm64
https://repo.anaconda.com/pkgs/r/noarch
package cache : /opt/anaconda3/pkgs
                /Users/keanu/.conda/pkgs
envs directories : /opt/anaconda3/envs
                  /Users/keanu/.conda/envs
temporary directory :
/var/folders/06/cw6fvh1n34v9h21qhm_pl16w0000gn/T
    platform : osx-arm64
    user-agent : conda/26.5.3 requests/2.34.2 CPython/3.14.6
Darwin/25.6.0 OSX/26.6.2 solver/libmamba conda-libmamba-solver/26.7.0
libmambapy/2.3.2 aau/0.8.1 c/. s/. e/.
    UID:GID : 501:20
    netrc file : None
offline mode : False
```

Task 2:

For the following tasks, I removed the 'Out [n]' terminal responses when pasting the commands into the 'Input' column, to save space and make the results more readable. However, I have numbered the tasks based off the order of appearance in the "Linear Algebra Equivalents" table.

Input	Output
# 1 In [1]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [2]: a = a_big.copy() In [3]: print(np.ndim(a))	2
# 2 In [4]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [5]: a = a_big.copy() In [6]: print(np.size(a))	240
# 3 In [7]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [8]: a = a_big.copy() In [9]: print(np.shape(a))	(24, 10)
# 4 In [10]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [11]: a = a_big.copy() In [12]: n = 1 In [13]: print(a.shape[n - 1])	24
# 5 In [14]: a = np.array([[1., 2., 3.],: [4., 5., 6.]]) In [15]: print(a)	[[1. 2. 3.] [4. 5. 6.]]
# 6 In [16]: a = np.array([[1., 2.,: [3., 4.]]) In [17]: b = np.array([[5., 6.,: [7., 8.]]) In [18]: c = np.array([[9., 10.,: [11., 12.]]) In [19]: d = np.array([[13., 14.,: [15., 16.]])	[[1. 2. 5. 6.] [3. 4. 7. 8.] [9. 10. 13. 14.] [11. 12. 15. 16.]]

In [20]: print(np.block([[a, b], [c, d]]))	
# 7 In [21]: a = np.array([4., 2., 7., 1., 9., 3.]) In [22]: print(a[-1])	3.0
# 8 In [23]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [24]: a = a_big.copy() In [25]: print(a[1, 4])	0.15
# 9 In [26]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [27]: a = a_big.copy() In [28]: print(a[1, :])	[0.11 0.12 0.13 0.14 0.15 0.16 0.17 0.18 0.19 0.2]
# 10 In [29]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [30]: a = a_big.copy() In [31]: print(a[0:5, :])	[[0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.1] [0.11 0.12 0.13 0.14 0.15 0.16 0.17 0.18 0.19 0.2] [0.21 0.22 0.23 0.24 0.25 0.26 0.27 0.28 0.29 0.3] [0.31 0.32 0.33 0.34 0.35 0.36 0.37 0.38 0.39 0.4] [0.41 0.42 0.43 0.44 0.45 0.46 0.47 0.48 0.49 0.5]]
# 11 In [32]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [33]: a = a_big.copy() In [34]: print(a[-5:])	[[1.91 1.92 1.93 1.94 1.95 1.96 1.97 1.98 1.99 2.] [2.01 2.02 2.03 2.04 2.05 2.06 2.07 2.08 2.09 2.1] [2.11 2.12 2.13 2.14 2.15 2.16 2.17 2.18 2.19 2.2] [2.21 2.22 2.23 2.24 2.25 2.26 2.27 2.28 2.29 2.3] [2.31 2.32 2.33 2.34 2.35 2.36 2.37 2.38 2.39 2.4]]
# 12 In [35]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [36]: a = a_big.copy() In [37]: print(a[0:3, 4:9])	[[0.05 0.06 0.07 0.08 0.09] [0.15 0.16 0.17 0.18 0.19] [0.25 0.26 0.27 0.28 0.29]]

<pre># 13 In [38]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [39]: a = a_big.copy() In [40]: print(a[np.ix_([1, 3, 4], [0, 2])))</pre>	<pre>[[0.11 0.13] [0.31 0.33] [0.41 0.43]]</pre>
<pre># 14 In [41]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [42]: a = a_big.copy() In [43]: print(a[2:21:2, :])</pre>	<pre>[[0.21 0.22 0.23 0.24 0.25 0.26 0.27 0.28 0.29 0.3] [0.41 0.42 0.43 0.44 0.45 0.46 0.47 0.48 0.49 0.5] [0.61 0.62 0.63 0.64 0.65 0.66 0.67 0.68 0.69 0.7] [0.81 0.82 0.83 0.84 0.85 0.86 0.87 0.88 0.89 0.9] [1.01 1.02 1.03 1.04 1.05 1.06 1.07 1.08 1.09 1.1] [1.21 1.22 1.23 1.24 1.25 1.26 1.27 1.28 1.29 1.3] [1.41 1.42 1.43 1.44 1.45 1.46 1.47 1.48 1.49 1.5] [1.61 1.62 1.63 1.64 1.65 1.66 1.67 1.68 1.69 1.7] [1.81 1.82 1.83 1.84 1.85 1.86 1.87 1.88 1.89 1.9] [2.01 2.02 2.03 2.04 2.05 2.06 2.07 2.08 2.09 2.1]]</pre>
<pre># 15 In [44]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [45]: a = a_big.copy() In [46]: print(a[:, :2, :])</pre>	<pre>[[0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.1] [0.21 0.22 0.23 0.24 0.25 0.26 0.27 0.28 0.29 0.3] [0.41 0.42 0.43 0.44 0.45 0.46 0.47 0.48 0.49 0.5] [0.61 0.62 0.63 0.64 0.65 0.66 0.67 0.68 0.69 0.7] [0.81 0.82 0.83 0.84 0.85 0.86 0.87 0.88 0.89 0.9] [1.01 1.02 1.03 1.04 1.05 1.06 1.07 1.08 1.09 1.1] [1.21 1.22 1.23 1.24 1.25 1.26 1.27 1.28 1.29 1.3] [1.41 1.42 1.43 1.44 1.45 1.46 1.47 1.48 1.49 1.5] [1.61 1.62 1.63 1.64 1.65</pre>

	<pre> 1.66 1.67 1.68 1.69 1.7] [1.81 1.82 1.83 1.84 1.85 1.86 1.87 1.88 1.89 1.9] [2.01 2.02 2.03 2.04 2.05 2.06 2.07 2.08 2.09 2.1] [2.21 2.22 2.23 2.24 2.25 2.26 2.27 2.28 2.29 2.3]]</pre>
<pre> # 16 In [47]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [48]: a = a_big.copy() In [49]: print(a[::-1, :])</pre>	<pre> [[2.31 2.32 2.33 2.34 2.35 2.36 2.37 2.38 2.39 2.4] [2.21 2.22 2.23 2.24 2.25 2.26 2.27 2.28 2.29 2.3] [2.11 2.12 2.13 2.14 2.15 2.16 2.17 2.18 2.19 2.2] [2.01 2.02 2.03 2.04 2.05 2.06 2.07 2.08 2.09 2.1] [1.91 1.92 1.93 1.94 1.95 1.96 1.97 1.98 1.99 2.] [1.81 1.82 1.83 1.84 1.85 1.86 1.87 1.88 1.89 1.9] [1.71 1.72 1.73 1.74 1.75 1.76 1.77 1.78 1.79 1.8] [1.61 1.62 1.63 1.64 1.65 1.66 1.67 1.68 1.69 1.7] [1.51 1.52 1.53 1.54 1.55 1.56 1.57 1.58 1.59 1.6] [1.41 1.42 1.43 1.44 1.45 1.46 1.47 1.48 1.49 1.5] [1.31 1.32 1.33 1.34 1.35 1.36 1.37 1.38 1.39 1.4] [1.21 1.22 1.23 1.24 1.25 1.26 1.27 1.28 1.29 1.3] [1.11 1.12 1.13 1.14 1.15 1.16 1.17 1.18 1.19 1.2] [1.01 1.02 1.03 1.04 1.05 1.06 1.07 1.08 1.09 1.1] [0.91 0.92 0.93 0.94 0.95 0.96 0.97 0.98 0.99 1.] [0.81 0.82 0.83 0.84 0.85 0.86 0.87 0.88 0.89 0.9] [0.71 0.72 0.73 0.74 0.75 0.76 0.77 0.78 0.79 0.8] [0.61 0.62 0.63 0.64 0.65 0.66 0.67 0.68 0.69 0.7] [0.51 0.52 0.53 0.54 0.55]</pre>

	<pre> 0.56 0.57 0.58 0.59 0.6] [0.41 0.42 0.43 0.44 0.45 0.46 0.47 0.48 0.49 0.5] [0.31 0.32 0.33 0.34 0.35 0.36 0.37 0.38 0.39 0.4] [0.21 0.22 0.23 0.24 0.25 0.26 0.27 0.28 0.29 0.3] [0.11 0.12 0.13 0.14 0.15 0.16 0.17 0.18 0.19 0.2] [0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.1]]</pre>
<pre> # 17 In [50]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [51]: a = a_big.copy() In [52]: print(a[np.r_[:len(a), 0]])</pre>	<pre> [[0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.1] [0.11 0.12 0.13 0.14 0.15 0.16 0.17 0.18 0.19 0.2] [0.21 0.22 0.23 0.24 0.25 0.26 0.27 0.28 0.29 0.3] [0.31 0.32 0.33 0.34 0.35 0.36 0.37 0.38 0.39 0.4] [0.41 0.42 0.43 0.44 0.45 0.46 0.47 0.48 0.49 0.5] [0.51 0.52 0.53 0.54 0.55 0.56 0.57 0.58 0.59 0.6] [0.61 0.62 0.63 0.64 0.65 0.66 0.67 0.68 0.69 0.7] [0.71 0.72 0.73 0.74 0.75 0.76 0.77 0.78 0.79 0.8] [0.81 0.82 0.83 0.84 0.85 0.86 0.87 0.88 0.89 0.9] [0.91 0.92 0.93 0.94 0.95 0.96 0.97 0.98 0.99 1.] [1.01 1.02 1.03 1.04 1.05 1.06 1.07 1.08 1.09 1.1] [1.11 1.12 1.13 1.14 1.15 1.16 1.17 1.18 1.19 1.2] [1.21 1.22 1.23 1.24 1.25 1.26 1.27 1.28 1.29 1.3] [1.31 1.32 1.33 1.34 1.35 1.36 1.37 1.38 1.39 1.4] [1.41 1.42 1.43 1.44 1.45 1.46 1.47 1.48 1.49 1.5] [1.51 1.52 1.53 1.54 1.55 1.56 1.57 1.58 1.59 1.6] [1.61 1.62 1.63 1.64 1.65</pre>

	<pre> 1.66 1.67 1.68 1.69 1.7] [1.71 1.72 1.73 1.74 1.75 1.76 1.77 1.78 1.79 1.8] [1.81 1.82 1.83 1.84 1.85 1.86 1.87 1.88 1.89 1.9] [1.91 1.92 1.93 1.94 1.95 1.96 1.97 1.98 1.99 2.] [2.01 2.02 2.03 2.04 2.05 2.06 2.07 2.08 2.09 2.1] [2.11 2.12 2.13 2.14 2.15 2.16 2.17 2.18 2.19 2.2] [2.21 2.22 2.23 2.24 2.25 2.26 2.27 2.28 2.29 2.3] [2.31 2.32 2.33 2.34 2.35 2.36 2.37 2.38 2.39 2.4] [0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.1]]</pre>
<pre> # 18 In [53]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [54]: a = a_big.copy() In [55]: print(a.transpose())</pre>	<pre> [[0.01 0.11 0.21 0.31 0.41 0.51 0.61 0.71 0.81 0.91 1.01 1.11 1.21 1.31 1.41 1.51 1.61 1.71 1.81 1.91 2.01 2.11 2.21 2.31] [0.02 0.12 0.22 0.32 0.42 0.52 0.62 0.72 0.82 0.92 1.02 1.12 1.22 1.32 1.42 1.52 1.62 1.72 1.82 1.92 2.02 2.12 2.22 2.32] [0.03 0.13 0.23 0.33 0.43 0.53 0.63 0.73 0.83 0.93 1.03 1.13 1.23 1.33 1.43 1.53 1.63 1.73 1.83 1.93 2.03 2.13 2.23 2.33] [0.04 0.14 0.24 0.34 0.44 0.54 0.64 0.74 0.84 0.94 1.04 1.14 1.24 1.34 1.44 1.54 1.64 1.74 1.84 1.94 2.04 2.14 2.24 2.34] [0.05 0.15 0.25 0.35 0.45 0.55 0.65 0.75 0.85 0.95 1.05 1.15 1.25 1.35 1.45 1.55 1.65 1.75 1.85 1.95 2.05 2.15 2.25 2.35] [0.06 0.16 0.26 0.36 0.46 0.56 0.66 0.76 0.86 0.96 1.06]</pre>

	<pre> 1.16 1.26 1.36 1.46 1.56 1.66 1.76 1.86 1.96 2.06 2.16 2.26 2.36] [0.07 0.17 0.27 0.37 0.47 0.57 0.67 0.77 0.87 0.97 1.07 1.17 1.27 1.37 1.47 1.57 1.67 1.77 1.87 1.97 2.07 2.17 2.27 2.37] [0.08 0.18 0.28 0.38 0.48 0.58 0.68 0.78 0.88 0.98 1.08 1.18 1.28 1.38 1.48 1.58 1.68 1.78 1.88 1.98 2.08 2.18 2.28 2.38] [0.09 0.19 0.29 0.39 0.49 0.59 0.69 0.79 0.89 0.99 1.09 1.19 1.29 1.39 1.49 1.59 1.69 1.79 1.89 1.99 2.09 2.19 2.29 2.39] [0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1. 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 2. 2.1 2.2 2.3 2.4]]</pre>
<pre> # 19 In [56]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [57]: a = a_big.copy() In [58]: print(a.conj().transpose())</pre>	<pre> [[[0.01 0.11 0.21 0.31 0.41 0.51 0.61 0.71 0.81 0.91 1.01 1.11 1.21 1.31 1.41 1.51 1.61 1.71 1.81 1.91 2.01 2.11 2.21 2.31] [0.02 0.12 0.22 0.32 0.42 0.52 0.62 0.72 0.82 0.92 1.02 1.12 1.22 1.32 1.42 1.52 1.62 1.72 1.82 1.92 2.02 2.12 2.22 2.32] [0.03 0.13 0.23 0.33 0.43 0.53 0.63 0.73 0.83 0.93 1.03 1.13 1.23 1.33 1.43 1.53 1.63 1.73 1.83 1.93 2.03 2.13 2.23 2.33] [0.04 0.14 0.24 0.34 0.44 0.54 0.64 0.74 0.84 0.94 1.04 1.14 1.24 1.34 1.44 1.54 1.64 1.74 1.84 1.94 2.04 2.14 2.24 2.34] [0.05 0.15 0.25 0.35 0.45</pre>

	<pre> 0.55 0.65 0.75 0.85 0.95 1.05 1.15 1.25 1.35 1.45 1.55 1.65 1.75 1.85 1.95 2.05 2.15 2.25 2.35] [0.06 0.16 0.26 0.36 0.46 0.56 0.66 0.76 0.86 0.96 1.06 1.16 1.26 1.36 1.46 1.56 1.66 1.76 1.86 1.96 2.06 2.16 2.26 2.36] [0.07 0.17 0.27 0.37 0.47 0.57 0.67 0.77 0.87 0.97 1.07 1.17 1.27 1.37 1.47 1.57 1.67 1.77 1.87 1.97 2.07 2.17 2.27 2.37] [0.08 0.18 0.28 0.38 0.48 0.58 0.68 0.78 0.88 0.98 1.08 1.18 1.28 1.38 1.48 1.58 1.68 1.78 1.88 1.98 2.08 2.18 2.28 2.38] [0.09 0.19 0.29 0.39 0.49 0.59 0.69 0.79 0.89 0.99 1.09 1.19 1.29 1.39 1.49 1.59 1.69 1.79 1.89 1.99 2.09 2.19 2.29 2.39] [0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1. 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 2. 2.1 2.2 2.3 2.4]]</pre>
<pre> # 20 In [59]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]]) In [60]: b = np.array([[2., 0., 0., 0., 0.], ...: [0., 3., 0., 0., 0.]])</pre>	<pre> [[12. 3. 0. 0. 0.] [2. 15. 4. 0. 0.] [0. 3. 16. 5. 0.] [0. 0. 4. 15. 6.] [0. 0. 0. 5. 12.]]</pre>

<pre>: [0., 0., 4., 0., 0.],: [0., 0., 0., 5., 0.],: [0., 0., 0., 0., 6.]] In [61]: print(a @ b) </pre>	
<pre> # 21 In [62]: a = np.array([[6., 1., 0., 0., 0.],: [1., 5., 1., 0., 0.],: [0., 1., 4., 1., 0.],: [0., 0., 1., 3., 1.],: [0., 0., 0., 1., 2.]]) In [63]: b = np.array([[2., 0., 0., 0., 0.],: [0., 3., 0., 0., 0.],: [0., 0., 4., 0., 0.],: [0., 0., 0., 5., 0.],: [0., 0., 0., 0., 6.]]) In [64]: print(a * b) </pre>	<pre> [[12. 0. 0. 0. 0.] [0. 15. 0. 0. 0.] [0. 0. 16. 0. 0.] [0. 0. 0. 15. 0.] [0. 0. 0. 0. 12.]] </pre>
<pre> # 22 In [65]: a = np.array([[6., 1., 1., 1., 1.],: [1., 5., 1., 1., 1.],: [1., 1., 4., 1., 1.],: [1., 1., 1., 3., 1.],: [1., 1., 1., 1., 2.]]) In [66]: b = np.array([[2., 1., 1., 1., 1.],: [1., 3., 1., </pre>	<pre> [[3. 1. 1. 1. 1.] [1. 1.66666667 1. 1. 1.] [1. 1. 1. 1. 1.] [1. 1. 1. 0.6 1.] [1. 1. 1. 1. 0.33333333]] </pre>


```
# 25
```

```
In [73]: a_big = np.arange(1, 241,  
dtype=float).reshape(24, 10) / 100
```

```
In [74]: a = a_big.copy()
```

```
In [75]: print(np.nonzero(a > 0.5))
```

```
(array([ 5,  5,  5,  5,  5,  
5,  5,  5,  5,  5,  6,  6,  
6,  6,  6,  6,  6,  
        6,  6,  6,  7,  7,  
7,  7,  7,  7,  7,  7,  7,  
7,  8,  8,  8,  8,  
        8,  8,  8,  8,  8,  
8,  9,  9,  9,  9,  9,  9,  
9,  9,  9,  9, 10,  
        10, 10, 10, 10, 10,  
10, 10, 10, 10, 11, 11, 11,  
11, 11, 11, 11, 11,  
        11, 11, 12, 12, 12,  
12, 12, 12, 12, 12, 12, 12,  
13, 13, 13, 13, 13,  
        13, 13, 13, 13, 13,  
14, 14, 14, 14, 14, 14, 14,  
14, 14, 14, 15, 15,  
        15, 15, 15, 15, 15,  
15, 15, 15, 16, 16, 16, 16,  
16, 16, 16, 16, 16,  
        16, 17, 17, 17, 17,  
17, 17, 17, 17, 17, 17, 18,  
18, 18, 18, 18, 18,  
        18, 18, 18, 18, 19,  
19, 19, 19, 19, 19, 19, 19,  
19, 19, 20, 20, 20,  
        20, 20, 20, 20, 20,  
20, 20, 21, 21, 21, 21, 21,  
21, 21, 21, 21, 21,  
        22, 22, 22, 22, 22,  
22, 22, 22, 22, 22, 23, 23,  
23, 23, 23, 23, 23,  
        23, 23, 23]),  
array([0, 1, 2, 3, 4, 5, 6,  
7, 8, 9, 0, 1, 2, 3, 4, 5, 6,  
7, 8, 9, 0, 1,  
        2, 3, 4, 5, 6, 7, 8,  
9, 0, 1, 2, 3, 4, 5, 6, 7, 8,  
9, 0, 1, 2, 3,  
        4, 5, 6, 7, 8, 9, 0,  
1, 2, 3, 4, 5, 6, 7, 8, 9, 0,  
1, 2, 3, 4, 5,  
        6, 7, 8, 9, 0, 1, 2,
```

	<pre> 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9))) </pre>
<pre> # 26 In [76]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]]) In [77]: v = np.array([0.2, 0.8, 0.4, 0.9, 0.6]) In [78]: print(a[:, np.nonzero(v > 0.5)[0]]) </pre>	<pre> [[1. 0. 0.] [5. 0. 0.] [1. 1. 0.] [0. 3. 1.] [0. 1. 2.]] </pre>
<pre> # 27 In [79]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]]) In [80]: v = np.array([0.2, 0.8, </pre>	<pre> [[1. 0. 0.] [5. 0. 0.] [1. 1. 0.] [0. 3. 1.] [0. 1. 2.]] </pre>

<pre>0.4, 0.9, 0.6]) In [81]: print(a[:, v.T > 0.5])</pre>	
<pre># 28 In [82]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [83]: a = a_big.copy() In [84]: a[a < 0.5] = 0 In [85]: print(a)</pre>	<pre>[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0.5 0.51 0.52 0.53 0.54 0.55 0.56 0.57 0.58 0.59 0.6] [0.61 0.62 0.63 0.64 0.65 0.66 0.67 0.68 0.69 0.7] [0.71 0.72 0.73 0.74 0.75 0.76 0.77 0.78 0.79 0.8] [0.81 0.82 0.83 0.84 0.85 0.86 0.87 0.88 0.89 0.9] [0.91 0.92 0.93 0.94 0.95 0.96 0.97 0.98 0.99 1.] [1.01 1.02 1.03 1.04 1.05 1.06 1.07 1.08 1.09 1.1] [1.11 1.12 1.13 1.14 1.15 1.16 1.17 1.18 1.19 1.2] [1.21 1.22 1.23 1.24 1.25 1.26 1.27 1.28 1.29 1.3] [1.31 1.32 1.33 1.34 1.35 1.36 1.37 1.38 1.39 1.4] [1.41 1.42 1.43 1.44 1.45 1.46 1.47 1.48 1.49 1.5] [1.51 1.52 1.53 1.54 1.55 1.56 1.57 1.58 1.59 1.6] [1.61 1.62 1.63 1.64 1.65 1.66 1.67 1.68 1.69 1.7] [1.71 1.72 1.73 1.74 1.75 1.76 1.77 1.78 1.79 1.8] [1.81 1.82 1.83 1.84 1.85 1.86 1.87 1.88 1.89 1.9] [1.91 1.92 1.93 1.94 1.95 1.96 1.97 1.98 1.99 2.] [2.01 2.02 2.03 2.04 2.05 2.06 2.07 2.08 2.09 2.1]]</pre>

	<pre> [2.11 2.12 2.13 2.14 2.15 2.16 2.17 2.18 2.19 2.2] [2.21 2.22 2.23 2.24 2.25 2.26 2.27 2.28 2.29 2.3] [2.31 2.32 2.33 2.34 2.35 2.36 2.37 2.38 2.39 2.4]]</pre>
<pre> # 29 In [86]: a_big = np.arange(1, 241, dtype=float).reshape(24, 10) / 100 In [87]: a = a_big.copy() In [88]: print(a * (a > 0.5))</pre>	<pre> [[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0. 0. 0. 0. 0.] [0.51 0.52 0.53 0.54 0.55 0.56 0.57 0.58 0.59 0.6] [0.61 0.62 0.63 0.64 0.65 0.66 0.67 0.68 0.69 0.7] [0.71 0.72 0.73 0.74 0.75 0.76 0.77 0.78 0.79 0.8] [0.81 0.82 0.83 0.84 0.85 0.86 0.87 0.88 0.89 0.9] [0.91 0.92 0.93 0.94 0.95 0.96 0.97 0.98 0.99 1.] [1.01 1.02 1.03 1.04 1.05 1.06 1.07 1.08 1.09 1.1] [1.11 1.12 1.13 1.14 1.15 1.16 1.17 1.18 1.19 1.2] [1.21 1.22 1.23 1.24 1.25 1.26 1.27 1.28 1.29 1.3] [1.31 1.32 1.33 1.34 1.35 1.36 1.37 1.38 1.39 1.4] [1.41 1.42 1.43 1.44 1.45 1.46 1.47 1.48 1.49 1.5] [1.51 1.52 1.53 1.54 1.55 1.56 1.57 1.58 1.59 1.6] [1.61 1.62 1.63 1.64 1.65 1.66 1.67 1.68 1.69 1.7] [1.71 1.72 1.73 1.74 1.75 1.76 1.77 1.78 1.79 1.8] [1.81 1.82 1.83 1.84 1.85 1.86 1.87 1.88 1.89 1.9]]</pre>

	<pre> [3. 3. 3. 3. 3. 3. 3. 3. 3. 3.] [3. 3. 3. 3. 3. 3. 3. 3. 3. 3.] [3. 3. 3. 3. 3. 3. 3. 3. 3. 3.] [3. 3. 3. 3. 3. 3. 3. 3. 3. 3.] [3. 3. 3. 3. 3. 3. 3. 3. 3. 3.] [3. 3. 3. 3. 3. 3. 3. 3. 3. 3.] [3. 3. 3. 3. 3. 3. 3. 3. 3. 3.] [3. 3. 3. 3. 3. 3. 3. 3. 3. 3.] </pre>
<pre> # 31 In [93]: x = np.array([[1., 2.], ...: [3., 4.]]) In [94]: y = x.copy() In [95]: print(y) </pre>	<pre> [[1. 2.] [3. 4.]] </pre>
<pre> # 32 In [96]: x = np.array([[1., 2., 3.], ...: [4., 5., 6.], ...: [7., 8., 9.]]) In [97]: y = x[1, :].copy() In [98]: print(y) </pre>	<pre> [4. 5. 6.] </pre>
<pre> # 33 In [99]: x = np.array([[1., 2., 3.], ...: [4., 5., 6.]]) In [100]: y = x.flatten() In [101]: print(y) </pre>	<pre> [1. 2. 3. 4. 5. 6.] </pre>
<pre> # 34 In [102]: print(np.arange(1., 11.)) </pre>	<pre> [1. 2. 3. 4. 5. 6. 7. 8. 9. 10.] </pre>
<pre> # 35 In [103]: print(np.arange(10.)) </pre>	<pre> [0. 1. 2. 3. 4. 5. 6. 7. 8. 9.] </pre>

<pre># 36 In [104]: print(np.arange(1., 11.)[:, np.newaxis])</pre>	<pre>[[1.] [2.] [3.] [4.] [5.] [6.] [7.] [8.] [9.] [10.]]</pre>
<pre># 37 In [105]: print(np.zeros((3, 4)))</pre>	<pre>[[0. 0. 0. 0.] [0. 0. 0. 0.] [0. 0. 0. 0.]]</pre>
<pre># 38 In [106]: print(np.zeros((3, 4, 5)))</pre>	<pre>[[[0. 0. 0. 0. 0.] [0. 0. 0. 0. 0.] [0. 0. 0. 0. 0.] [0. 0. 0. 0. 0.]] [[0. 0. 0. 0. 0.] [0. 0. 0. 0. 0.] [0. 0. 0. 0. 0.] [0. 0. 0. 0. 0.]] [[0. 0. 0. 0. 0.] [0. 0. 0. 0. 0.] [0. 0. 0. 0. 0.] [0. 0. 0. 0. 0.]]]</pre>
<pre># 39 In [107]: print(np.ones((3, 4)))</pre>	<pre>[[1. 1. 1. 1.] [1. 1. 1. 1.] [1. 1. 1. 1.]]</pre>
<pre># 40 In [108]: print(np.eye(3))</pre>	<pre>[[1. 0. 0.] [0. 1. 0.] [0. 0. 1.]]</pre>
<pre># 41 In [109]: a = np.array([[6., 1., 0.], ...: [1., 5., 1.], ...: [0., 1., 4.]]) In [110]: print(np.diag(a))</pre>	<pre>[6. 5. 4.]</pre>

<pre># 42 In [111]: v = np.array([0.2, 0.8, 0.4, 0.9, 0.6]) In [112]: print(np.diag(v, 0))</pre>	<pre>[[0.2 0. 0. 0. 0.] [0. 0.8 0. 0. 0.] [0. 0. 0.4 0. 0.] [0. 0. 0. 0.9 0.] [0. 0. 0. 0. 0.6]]</pre>
<pre># 43 In [113]: rng = default_rng(42) In [114]: print(rng.random((3, 4)))</pre>	<pre>[[0.77395605 0.43887844 0.85859792 0.69736803] [0.09417735 0.97562235 0.7611397 0.78606431] [0.12811363 0.45038594 0.37079802 0.92676499]]</pre>
<pre># 44 In [115]: print(np.linspace(1, 3, 4))</pre>	<pre>[1. 1.66666667 2.33333333 3.]</pre>
<pre># 45 In [116]: print(np.mgrid[0:9., 0:6.])</pre>	<pre>[[[0. 0. 0. 0. 0. 0.] [1. 1. 1. 1. 1. 1.] [2. 2. 2. 2. 2. 2.] [3. 3. 3. 3. 3. 3.] [4. 4. 4. 4. 4. 4.] [5. 5. 5. 5. 5. 5.] [6. 6. 6. 6. 6. 6.] [7. 7. 7. 7. 7. 7.] [8. 8. 8. 8. 8. 8.]] [[0. 1. 2. 3. 4. 5.] [0. 1. 2. 3. 4. 5.] [0. 1. 2. 3. 4. 5.] [0. 1. 2. 3. 4. 5.] [0. 1. 2. 3. 4. 5.] [0. 1. 2. 3. 4. 5.] [0. 1. 2. 3. 4. 5.] [0. 1. 2. 3. 4. 5.] [0. 1. 2. 3. 4. 5.]]]</pre>
<pre># 46 In [117]: print(np.ix_(np.r_[0:9.], np.r_[0:6.]))</pre>	<pre>(array([[0.], [1.], [2.], [3.], [4.], [5.], [6.], [7.],</pre>

	<code>[8.]])</code> , <code>array([[0., 1., 2., 3., 4., 5.]])</code>
# 47 In [118]: <code>print(np.meshgrid([1, 2, 4], [2, 4, 5]))</code>	<code>(array([[1, 2, 4], [1, 2, 4], [1, 2, 4]]), array([[2, 2, 2], [4, 4, 4], [5, 5, 5]]))</code>
# 48 In [119]: <code>print(np.ix_([1, 2, 4], [2, 4, 5]))</code>	<code>(array([[1], [2], [4]]), array([[2, 4, 5]]))</code>
# 49 In [120]: <code>a = np.array([[1., 2.], ...: [3., 4.]])</code> In [121]: <code>m = 2</code> In [122]: <code>n = 3</code> In [123]: <code>print(np.tile(a, (m, n)))</code>	<code>[[1. 2. 1. 2. 1. 2.] [3. 4. 3. 4. 3. 4.] [1. 2. 1. 2. 1. 2.] [3. 4. 3. 4. 3. 4.]]</code>
# 50 In [124]: <code>a = np.array([[1., 2.], ...: [3., 4.]])</code> In [125]: <code>b = np.array([[5., 6.], ...: [7., 8.]])</code> In [126]: <code>print(np.concatenate((a, b), axis=1))</code>	<code>[[1. 2. 5. 6.] [3. 4. 7. 8.]]</code>
# 51 In [127]: <code>a = np.array([[1., 2.], ...: [3., 4.]])</code> In [128]: <code>b = np.array([[5., 6.], ...: [7., 8.]])</code> In [129]: <code>print(np.concatenate((a, b)))</code>	<code>[[1. 2.] [3. 4.] [5. 6.] [7. 8.]]</code>
# 52 In [130]: <code>a = np.array([[6., 1., 4.], ...: [2., 9., 3.], ...: [5., 7., 8.]])</code> In [131]: <code>print(a.max())</code>	9.0

<pre># 53 In [132]: a = np.array([[6., 1., 4.], ...: [2., 9., 3.], ...: [5., 7., 8.]]) In [133]: print(a.max(0))</pre>	<pre>[6. 9. 8.]</pre>
<pre># 54 In [134]: a = np.array([[6., 1., 4.], ...: [2., 9., 3.], ...: [5., 7., 8.]]) In [135]: print(a.max(1))</pre>	<pre>[6. 9. 8.]</pre>
<pre># 55 In [136]: a = np.array([[1., 5.], ...: [7., 2.]]) In [137]: b = np.array([[4., 3.], ...: [6., 8.]]) In [138]: print(np.maximum(a, b))</pre>	<pre>[[4. 5.] [7. 8.]]</pre>
<pre># 56 In [139]: v = np.array([0.2, 0.8, 0.4, 0.9, 0.6]) In [140]: print(np.linalg.norm(v))</pre>	<pre>1.4177446878757827</pre>
<pre># 57 In [141]: a = np.array([[True, False, True], ...: [False, True, True]]) In [142]: b = np.array([[True, True, False], ...: [False, True, False]]) In [143]: print(np.logical_and(a, b))</pre>	<pre>[[True False False] [False True False]]</pre>
<pre># 58 In [144]: a = np.array([[True, False, True], ...: [False, True, True]]) In [145]: b = np.array([[True, True, False],</pre>	<pre>[[True True True] [False True True]]</pre>

<pre> ...: [False, True, False]]) In [146]: print(np.logical_or(a, b)) </pre>	
<pre> # 59 In [147]: a = np.array([[1, 2, 3], ...: [4, 5, 6]]) In [148]: b = np.array([[3, 1, 2], ...: [1, 4, 7]]) In [149]: print(a & b) </pre>	<pre> [[1 0 2] [0 4 6]] </pre>
<pre> # 60 In [150]: a = np.array([[1, 2, 3], ...: [4, 5, 6]]) In [151]: b = np.array([[3, 1, 2], ...: [1, 4, 7]]) In [152]: print(a b) </pre>	<pre> [[3 3 3] [5 5 7]] </pre>
<pre> # 61 In [153]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]]) In [154]: print(scipy.linalg.inv(a)) </pre>	<pre> [[0.17276423 -0.03658537 0.0101626 -0.00406504 0.00203252] [-0.03658537 0.2195122 -0.06097561 0.02439024 -0.01219512] [0.0101626 -0.06097561 0.29471545 -0.11788618 0.05894309] [-0.00406504 0.02439024 -0.11788618 0.44715447 -0.22357724] [0.00203252 -0.01219512 0.05894309 -0.22357724 0.61178862]] </pre>
<pre> # 62 In [155]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]]) </pre>	<pre> [[0.17276423 -0.03658537 0.0101626 -0.00406504 0.00203252] [-0.03658537 0.2195122 -0.06097561 0.02439024 -0.01219512] [0.0101626 -0.06097561 0.29471545 -0.11788618 0.05894309] [-0.00406504 0.02439024 -0.11788618 0.44715447 </pre>

<pre>In [156]: print(scipy.linalg.pinv(a))</pre>	<pre>-0.22357724] [0.00203252 -0.01219512 0.05894309 -0.22357724 0.61178862]]</pre>
<pre># 63 In [157]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]]) In [158]: print(np.linalg.matrix_rank(a))</pre>	<pre>5</pre>
<pre># 64 In [159]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]]) In [160]: b = np.array([1., 2., 3., 4., 5.]) In [161]: print(scipy.linalg.solve(a, b))</pre>	<pre>[0.12398374 0.25609756 0.59552846 0.36178862 2.31910569]</pre>
<pre># 65 In [162]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]])</pre>	<pre>[0.12398374 0.25609756 0.59552846 0.36178862 2.31910569]</pre>

<pre> In [163]: b = np.array([1., 2., 3., 4., 5.]) In [164]: x = scipy.linalg.solve(a.T, b.T).T In [165]: print(x) </pre>	
<pre> # 66 In [166]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]]) In [167]: U, S, Vh = scipy.linalg.svd(a) In [168]: V = Vh.T In [169]: print("U:") In [170]: print(U) In [171]: print("S:") In [172]: print(S) In [173]: print("V:") In [174]: print(V) </pre>	<pre> U: [[-0.77704718 0.54249497 -0.30151134 0.10387993 0.0140272] [-0.57979962 -0.42980092 0.60302269 -0.33321906 -0.06657528] [-0.23537429 -0.63177876 -0.30151134 0.63177876 0.23537429] [-0.06657528 -0.33321906 -0.60302269 -0.42980092 -0.57979962] [-0.0140272 -0.10387993 -0.30151134 -0.54249497 0.77704718]] S: [6.74615755 5.20773289 4. 2.79226711 1.25384245] V: [[-0.77704718 0.54249497 -0.30151134 0.10387993 0.0140272] [-0.57979962 -0.42980092 0.60302269 -0.33321906 -0.06657528] [-0.23537429 -0.63177876 -0.30151134 0.63177876 0.23537429] [-0.06657528 -0.33321906 -0.60302269 -0.42980092 -0.57979962] [-0.0140272 -0.10387993 -0.30151134 -0.54249497 0.77704718]] </pre>
<pre> # 67 In [175]: a = np.array([[6., 1., 0., 0., 0.], </pre>	<pre> [[2.44948974 0.40824829 0. 0. 0.] [0. 2.19848433 </pre>

<pre>: [1., 5., 1., 0., 0.],: [0., 1., 4., 1., 0.],: [0., 0., 1., 3., 1.],: [0., 0., 0., 1., 2.]])) In [176]: print(scipy.linalg.cholesky(a)) </pre>	<pre> 0.45485883 0. 0.] [0. 0. 1.94758914 0.51345532 0.] [0. 0. 0. 1.65419577 0.60452337] [0. 0. 0. 0. 1.27849579]] </pre>
<pre> # 68 In [177]: a = np.array([[6., 1., 0., 0., 0.],: [1., 5., 1., 0., 0.],: [0., 1., 4., 1., 0.],: [0., 0., 1., 3., 1.],: [0., 0., 0., 1., 2.]])) In [178]: D, V = scipy.linalg.eig(a) In [179]: print("Eigenvalues:") In [180]: print(D) In [181]: print("Eigenvectors:") In [182]: print(V) </pre>	<pre> Eigenvalues: [6.74615755+0.j 5.20773289+0.j 4. +0.j 2.79226711+0.j 1.25384245+0.j] Eigenvectors: [[0.77704718 -0.54249497 -0.30151134 0.10387993 0.0140272] [0.57979962 0.42980092 0.60302269 -0.33321906 -0.06657528] [0.23537429 0.63177876 -0.30151134 0.63177876 0.23537429] [0.06657528 0.33321906 -0.60302269 -0.42980092 -0.57979962] [0.0140272 0.10387993 -0.30151134 -0.54249497 0.77704718]] </pre>
<pre> # 69 In [183]: a = np.array([[6., 1., 0., 0., 0.],: [1., 5., 1., 0., 0.],: [0., 1., 4., 1., 0.],: [0., 0., 1., 3., 1.],: [0., 0., 0., 1., 2.]])) In [184]: b = np.array([[2., 0., </pre>	<pre> Eigenvalues: [3.11806282+0.j 1.67401751+0.j 1. +0.j 0.58359571+0.j 0.22432396+0.j] Eigenvectors: [[9.72869450e-01 -3.29786190e-01 1.09764260e-01 -3.53936551e-02 8.15258886e-03] [2.29719413e-01 </pre>

<pre> 0., 0., 0.], ...: [0., 3., 0., 0., 0.], ...: [0., 0., 4., 0., 0.], ...: [0., 0., 0., 5., 0.], ...: [0., 0., 0., 0., 6.]]) In [185]: D, V = scipy.linalg.eig(a, b) In [186]: print("Eigenvalues:") In [187]: print(D) In [188]: print("Eigenvectors:") In [189]: print(V) </pre>	<pre> 8.74581423e-01 -4.39057040e-01 1.71050760e-01 -4.52578912e-02] [2.73721654e-02 3.49072934e-01 7.68349820e-01 -5.20386672e-01 1.87679580e-01] [2.18444949e-03 6.65436606e-02 4.39057040e-01 6.95714202e-01 -5.37056324e-01] [1.30739778e-04 8.27235098e-03 1.09764260e-01 4.63323199e-01 8.21116398e-01]] </pre>
<pre> # 70 In [190]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]]) In [191]: D, V = eigs(a, k=3) In [192]: print("Eigenvalues:") In [193]: print(D) In [194]: print("Eigenvectors:") In [195]: print(V) </pre>	<pre> Eigenvalues: [6.74615755+0.j 4. +0.j 5.20773289+0.j] Eigenvectors: [[0.77704718+0.j 0.30151134+0.j -0.54249497+0.j] [0.57979962+0.j -0.60302269+0.j 0.42980092+0.j] [0.23537429+0.j 0.30151134+0.j 0.63177876+0.j] [0.06657528+0.j 0.60302269+0.j 0.33321906+0.j] [0.0140272 +0.j 0.30151134+0.j 0.10387993+0.j]] </pre>
<pre> # 71 In [196]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], </pre>	<pre> Q: [[-0.98639392 0.16089772 -0.03258679 0.00820969 0.00310716] [-0.16439899 -0.96538629 </pre>

<pre> ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]] In [197]: Q, R = scipy.linalg.qr(a) In [198]: print("Q:") In [199]: print(Q) In [200]: print("R:") In [201]: print(R) </pre>	<pre> 0.19552072 -0.04925814 -0.01864298] [-0. -0.20528329 -0.94501682 0.23808101 0.09010775] [-0. -0. -0.26010181 -0.90306589 -0.34178801] [-0. -0. -0. -0.35397128 0.93525629]] R: [[-6.08276253 -1.80838886 -0.16439899 0. 0.] [0. -4.87131704 -1.78651946 -0.20528329 0.] [0. 0. -3.84464838 -1.72532225 -0.26010181] [0. 0. 0. -2.82508795 -1.61100845] [0. 0. 0. 0. 1.52872457]] </pre>
<pre> # 72 In [202]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]]) In [203]: P, L, U = scipy.linalg.lu(a) In [204]: print("P:") In [205]: print(P) In [206]: print("L:") In [207]: print(L) </pre>	<pre> P: [[1. 0. 0. 0. 0.] [0. 1. 0. 0. 0.] [0. 0. 1. 0. 0.] [0. 0. 0. 1. 0.] [0. 0. 0. 0. 1.]] L: [[1. 0. 0. 0. 0.] [0.16666667 1. 0. 0. 0.] [0. 0.20689655 1. 0. 0.] [0. 0. 0. 0.26363636 1. 0.] [0. 0. 0. </pre>

<pre> In [208]: print("U:") In [209]: print(U) </pre>	<pre> 0.3654485 1.]] U: [[6. 1. 0. 0. 0.] [0. 4.83333333 1. 0. 0.] [0. 0. 3.79310345 1. 0.] [0. 0. 0. 2.73636364 1.] [0. 0. 0. 0. 1.6345515]] </pre>
<pre> # 73 In [210]: a = np.array([[6., 1., 0., 0., 0.], ...: [1., 5., 1., 0., 0.], ...: [0., 1., 4., 1., 0.], ...: [0., 0., 1., 3., 1.], ...: [0., 0., 0., 1., 2.]]) In [211]: b = np.array([1., 2., 3., 4., 5.]) In [212]: x, info = cg(a, b) In [213]: print(x) In [214]: print(info) </pre>	<pre> [0.12398374 0.25609756 0.59552846 0.36178862 2.31910569] 0 </pre>
<pre> # 74 In [215]: a = np.array([4., 2., 7., 1., 9., 3.]) In [216]: print(np.fft.fft(a)) </pre>	<pre> [26. +0.j -2.5+2.59807621j -5.5-0.8660254j 14. +0.j -5.5+0.8660254j -2.5-2.59807621j] </pre>
<pre> # 75 In [217]: a = np.array([4., 2., 7., 1., 9., 3.]) In [218]: print(np.fft.ifft(a)) </pre>	<pre> [4.33333333+0.j -0.41666667-0.4330127j -0.91666667+0.14433757j 2.33333333+0.j -0.91666667-0.14433757j -0.41666667+0.4330127j] </pre>

<pre># 76 In [219]: a = np.array([[6., 1., 4.], ...: [2., 9., 3.], ...: [5., 7., 8.]]) In [220]: print(np.sort(a))</pre>	<pre>[[1. 4. 6.] [2. 3. 9.] [5. 7. 8.]]</pre>
<pre># 77 In [221]: a = np.array([[6., 1., 4.], ...: [2., 9., 3.], ...: [5., 7., 8.]]) In [222]: print(np.sort(a, axis=1))</pre>	<pre>[[1. 4. 6.] [2. 3. 9.] [5. 7. 8.]]</pre>
<pre># 78 In [223]: a = np.array([[6., 1., 4.], ...: [2., 9., 3.], ...: [5., 7., 8.]]) In [224]: I = np.argsort(a[:, 0]) In [225]: b = a[I, :] In [226]: print("I:") In [227]: print(I) In [228]: print("b:") In [229]: print(b)</pre>	<pre>I: [1 2 0] b: [[2. 9. 3.] [5. 7. 8.] [6. 1. 4.]]</pre>
<pre># 79 In [230]: Z = np.array([[1., 1.], ...: [1., 2.], ...: [1., 3.], ...: [1., 4.], ...: [1., 5.]]) In [231]: y = np.array([2., 4., 5., 8., 10.]) In [232]: x = scipy.linalg.lstsq(Z, y)[0] In [233]: print(x)</pre>	<pre>[-0.2 2.]</pre>
<pre># 80 In [234]: x = np.array([4., 2., 7., 1., 9., 3.]) In [235]: q = 2 In [236]:</pre>	<pre>[3.5 4. 5.5]</pre>

<pre>print(scipy.signal.resample(...: x, ...: int(np.ceil(len(x) / q)) ...:))</pre>	
<pre># 81 In [237]: a = np.array([4., 2., 7., 1., 9., 3., 4., 2.]) In [238]: print(np.unique(a))</pre>	<pre>[1. 2. 3. 4. 7. 9.]</pre>
<pre># 82 In [239]: a = np.array([[[1., 2., 3., 4., 5.]]) In [240]: print("Before squeeze:") In [241]: print(a) In [242]: print("Shape:", a.shape) In [243]: print("After squeeze:") In [244]: print(a.squeeze()) In [245]: print("Shape:", a.squeeze().shape)</pre>	<pre>Before squeeze: [[[1. 2. 3. 4. 5.]]) Shape: (1, 1, 5) After squeeze: [1. 2. 3. 4. 5.] Shape: (5,)</pre>

Task 3

```
In [1]: import matplotlib.pyplot as plt
```

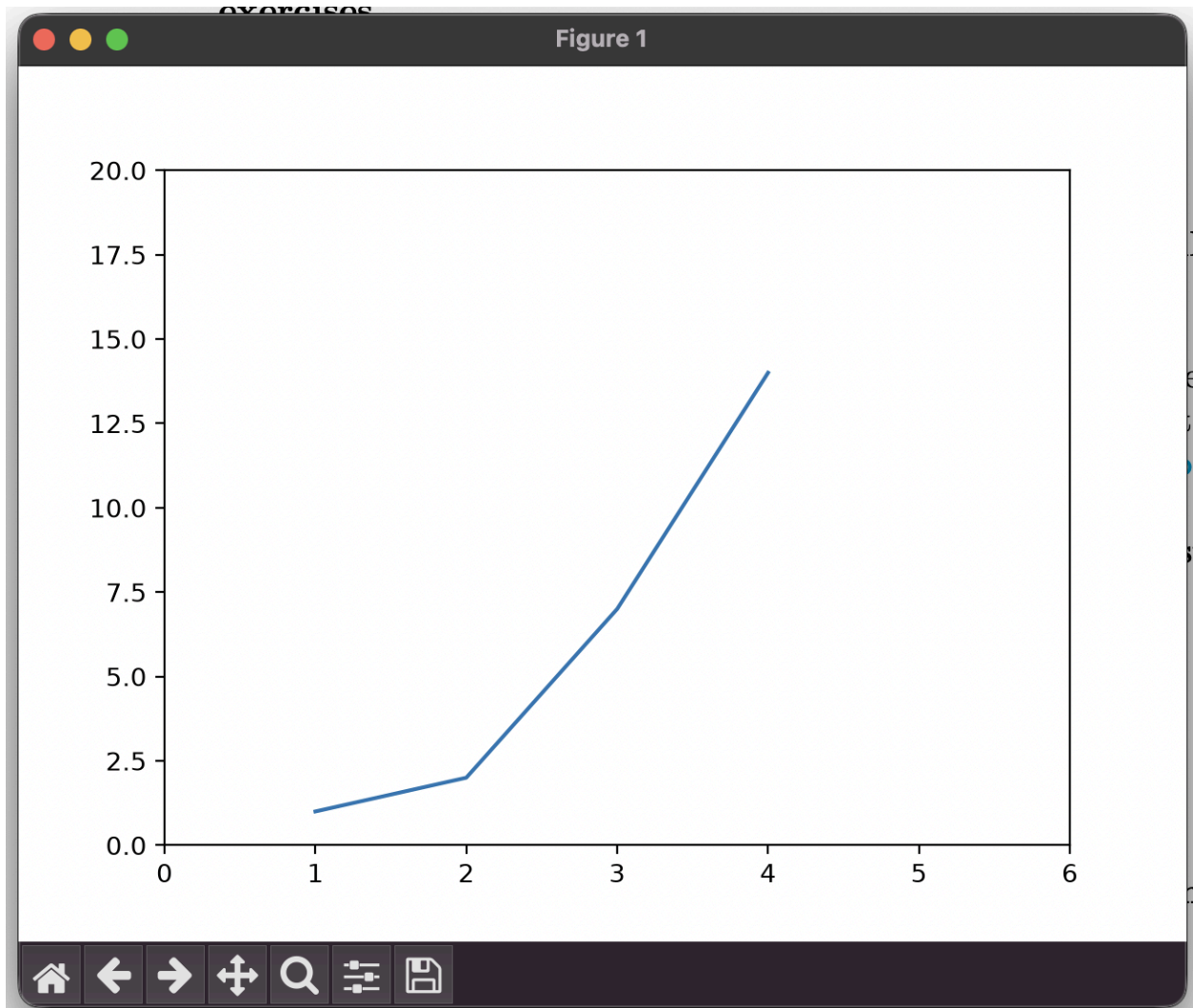
```
In [2]: plt.plot([1,2,3,4], [1,2,7,14])
```

```
Out[2]: [<matplotlib.lines.Line2D at 0x11e98d010>]
```

```
In [3]: plt.axis([0, 6, 0, 20])
```

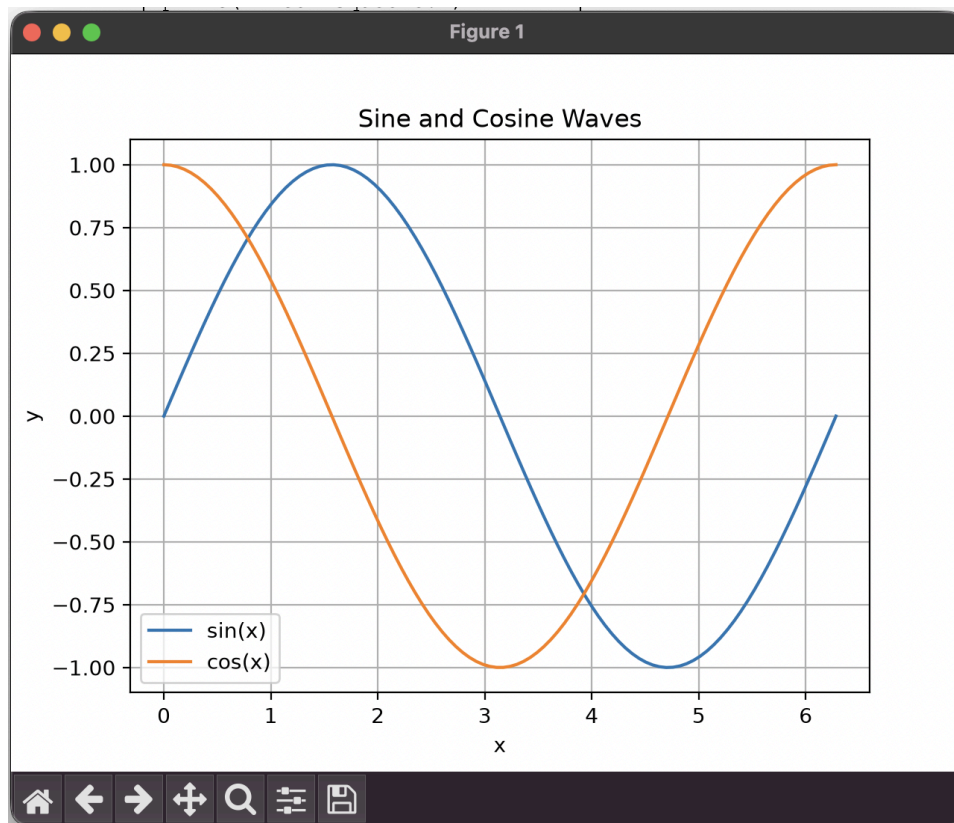
```
Out[3]: (np.float64(0.0), np.float64(6.0), np.float64(0.0),  
np.float64(20.0))
```

```
In [4]: plt.show()
```



Task 4

```
In [1]: import numpy as np
In [2]: import matplotlib.pyplot as plt
In [3]: x = np.linspace(0, 2 * np.pi, 100)
In [4]: y1 = np.sin(x)
In [5]: y2 = np.cos(x)
In [6]: plt.plot(x, y1, label="sin(x)")
Out[6]: [<matplotlib.lines.Line2D at 0x17f680c20>]
In [7]: plt.plot(x, y2, label="cos(x)")
Out[7]: [<matplotlib.lines.Line2D at 0x17f680ec0>]
In [8]: plt.title("Sine and Cosine Waves")
Out[8]: Text(0.5, 1.0, 'Sine and Cosine Waves')
In [9]: plt.xlabel("x")
Out[9]: Text(0.5, 47.266666666666666, 'x')
In [10]: plt.ylabel("y")
Out[10]: Text(43.444444444444443, 0.5, 'y')
In [11]: plt.legend()
Out[11]: <matplotlib.legend.Legend at 0x17f661400>
In [12]: plt.grid()
In [13]: plt.show()
```



Task 5

GitHub Account Name: kswart04

Task 6

Link to GitHub repository: <https://github.com/kswart04/comp-576-assignment-0>